

MODULE 1: SOFTWARE CONSTRUCTION 1.0

WEEK 1

Learning Outcomes:

After completing this course, you are expected to demonstrate the following:

1. Improve the existing program by using coding practices, and standards for security and exception handling to avoid unexpected errors.

A. Engage

Trivia



Figure 1.0

In the near future knowing how to code will be as necessary as knowing how to write is today. Yes, regardless of the field or career your students decide for their future.

B. Explore

- Video Titles:**
1. Coding Best Practices with Examples | Code Review Best Practices
 2. Clean Code_ Learn to write clean, maintainable and robust code
 3. Secure Coding Techniques
 4. Java Programming Tutorial Exception Handling

- YouTube Links:**
1. <https://www.youtube.com/watch?v=rrii6QRX8ZLs>
 2. <https://www.youtube.com/watch?v=5koPpYVa020>
 3. <https://www.youtube.com/watch?v=RTifWn01I2g>
 4. https://www.youtube.com/watch?v=K_-3OLkXkzY

C. Explain

Introduction

Coding Standards and Guidelines - Different modules specified in the design document are coded in the Coding phase according to the module specification. The main goal of the coding phase is to code from the design document prepared after the design phase through a high-level language and then to unit test this code.

Good software development organizations want their programmers to maintain to some well-defined and standard style of coding called coding standards. They usually make their own coding standards and guidelines depending on what suits their organization best and based on the types of software they develop. It is very important for the programmers to maintain the coding standards otherwise the code will be rejected during code review.

Purpose of Having Coding Standards - A coding standard gives a uniform appearance to the codes written by different engineers.

- It improves readability, and maintainability of the code and it reduces complexity also.
- It helps in code reuse and helps to detect error easily.
- It promotes sound programming practices and increases efficiency of the programmers.

D. Elaborate

Coding standards are given below:

1. Limited use of global - These rules tell about which types of data that can be declared global and the data that can't be.

2. Standard headers for different modules - For better understanding and maintenance of the code, the header of different modules should follow some standard format and information. The header format must contain below things that is being used in various companies:

- ✧ Name of the module
- ✧ Date of module creation
- ✧ Author of the module
- ✧ Modification history
- ✧ Synopsis of the module about what the module does
- ✧ Different functions supported in the module along with their input output parameters
- ✧ Global variables accessed or modified by the module

3. Naming conventions for local variables, global variables, constants and functions -

- ✓ Meaningful and understandable variables name help anyone to understand the reason of using it.
- ✓ Local variables should be named using camel case lettering starting with small letter (e.g. localData) whereas Global variables names should start with a capital letter (e.g. GlobalData). Constant names should be formed using capital letters only (e.g. CONSDATA).
- ✓ It is better to avoid the use of digits in variable names.
- ✓ The names of the function should be written in camel case starting with small letters.
- ✓ The name of the function must describe the reason of using the function clearly and briefly.

4. Indention - Proper indentation is very important to increase the readability of the code. For making the code readable, programmers should use White spaces properly. Some of the spacing conventions are given below:

- ❖ There must be a space after giving a comma between two function arguments.
- ❖ Each nested block should be properly indented and spaced.
- ❖ Proper Indentation should be there at the beginning and at the end of each block in the program.
- ❖ All braces should start from a new line and the code following the end of braces also start from a new line.

a) **Error return values and exception handling conventions** - All functions that encounters an error condition should either return a 0 or 1 for simplifying the debugging.

On the other hand, Coding guidelines give some general suggestions regarding the coding style that to be followed for the betterment of understandability and readability of the code. Some of the coding guidelines are given below:

5. Avoid using a coding style that is too difficult to understand - Code should be easily understandable. The complex code makes maintenance and debugging difficult and expensive.

6. Avoid using an identifier for multiple purposes - Each variable should be given a descriptive and meaningful name indicating the reason behind using it. This is not possible if an identifier is used for multiple purposes and thus it can lead to confusion to the reader. Moreover, it leads to more difficulty during future enhancements.

7. Code should be well documented - The code should be properly commented for understanding easily. Comments regarding the statements increase the understandability of the code.

8. Length of functions should not be very large - Lengthy functions are very difficult to understand. That's why functions should be small enough to carry out small work and lengthy functions should be broken into small ones for completing small tasks.

9. Try not to use GOTO statement - GOTO statement makes the program unstructured, thus it reduces the understandability of the program and also debugging becomes difficult.

Advantages of Coding Guidelines:

- Coding guidelines increase the efficiency of the software and reduces the development time.
- Coding guidelines help in detecting errors in the early phases, so it helps to reduce the extra cost incurred by the software project.
- If coding guidelines are maintained properly, then the software code increases readability and understandability thus it reduces the complexity of the code.
- It reduces the hidden cost for developing the software.

Secure Coding Best Practices

When you hire builders to construct a new home, you expect them to take every precaution to ensure once you move in, you won't find split beams, foundational errors, or holes in the walls. In the same way, software developers are expected to uphold secure coding standards to ensure they aren't leaving any vulnerabilities open for hackers to exploit. Secure coding standards govern the coding practices, techniques, and decisions that developers make while building software. They aim to ensure that developers write code that minimizes security vulnerabilities. Development tasks can typically be solved in many different ways, with varying levels of complexity. Some solutions are more secure than others, and secure coding standards encourage developers and software engineers to take the safest approach, even if it is not the fastest.

For example, secure coding guidelines often mandate a "default deny" approach to access permissions. Developers should create code that denies access to sensitive resources unless an individual can demonstrate that they are authorized to access it.

8 Secure Coding Best Practices

Checklist for secure coding practices that includes 14 areas to consider in your software development life cycle. Of those secure coding concepts, we're going to focus on the top eight secure coding best practices to help you protect against vulnerabilities.

1. Security by Design

Security needs to be a priority as you develop code, not an afterthought. Organizations may have competing priorities where software engineering and coding are concerned. Optimizing for security can conflict with optimizing for development speed. However, a "security by design" approach that puts security first tends to pay off in the long run, reducing the future cost of technical debt and risk mitigation. An analysis of your source code should be conducted throughout your SDLC, and security automation should be implemented.

2. Password Management

Passwords are a weak point in many software systems, which is why multi-factor authentication has become so widespread. Nevertheless, passwords are the most common security credential, and following coding best practices limits risk. You should require all passwords to be of adequate length and complexity to withstand any typical or common attacks. OWASP suggests several coding best practices for passwords, including:

- Storing only salted cryptographic hashes of passwords and never storing plain-text passwords.
- Enforcing password length and complexity requirements.
- Disable password entry after multiple incorrect login attempts.

3. Access Control

Take a “default deny” approach to sensitive data. Limit privileges and restrict access to secure data to only users who need it. Deny access to any user that cannot demonstrate authorization. Ensure that requests for sensitive information are checked to verify that the user is authorized to access it.

4. Error Handling and Logging

Software errors are often indicative of bugs, many of which cause vulnerabilities. Error handling and logging are two of the most useful techniques for minimizing their impact. Error handling attempts to catch errors in the code before they result in a catastrophic failure. Logging documents errors so that developers can diagnose and mitigate their cause. Documentation and logging of all failures, exceptions, and errors should be implemented on a trusted system to comply with secure coding standards.

5. System Configuration

Clear your system of any unnecessary components and ensure all working software is updated with current versions and patches. If you work in multiple environments, make sure you’re managing your development and production environments securely. Outdated software is a major source of vulnerabilities and security breaches. Software updates include patches that fix vulnerabilities, making regular updates one of the most vital, secure coding practices.

6. Threat Modeling

Document, locate, address, and validate are the four steps to threat modeling. To securely code, you need to examine your software for areas susceptible to increased threats of attack. Threat modeling is a multi-stage process that should be integrated into the software lifecycle from development, testing, and production.

7. Cryptographic Practices

Using quality modern cryptographic algorithms with keys stored in secure key vaults is a practice that increases the security of your code in the event of a breach.

8. Input Validation and Output Encoding

These secure coding standards are self-explanatory in that you need to identify all data inputs and sources and validate those classified as untrusted. You should utilize a standard routine for output encoding and input validation.

E. Evaluate

ASSESSMENT:

Instructions: Write your answer on the Answer Sheet (AS) provided in this module. (60 pts)

1. Enumeration

- a) Give the 9 coding standards (2 pts each)
- b) Give the 4 Advantages of Coding Guidelines 3 pts each)
- c) Give the 8 Secure Coding Practices (2 pts each)
- d) Give the 4 Spacing convention (2 pts each)
- e) Give the 3 coding practices for passwords (2 pts)

References:

1. Coding Standards and Guidelines
<https://www.geeksforgeeks.org/coding-standards-and-guidelines/>

2. Secure Coding Best Practices
<https://kirkpatrickprice.com/blog/secure-coding-best-practices/>

3. Coding Best Practices with Examples | Code Review Best Practices
<https://www.youtube.com/watch?v=rji6QRX8ZLs>

4. Clean Code_ Learn to write clean, maintainable and robust code
<https://www.youtube.com/watch?v=5koPpYVa020>

Facilitated By:		
Name	:	
MS Teams Account (email)	:	
Smart Phone Number	:	